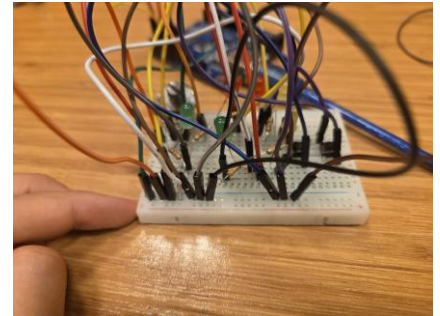
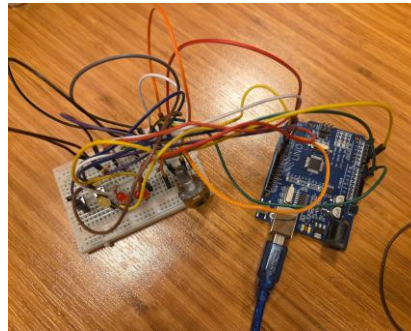
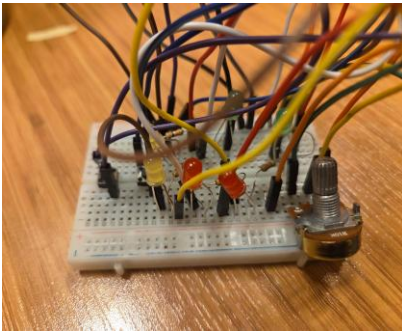


Lab 5 Report

1) Lab Content & What We Did (Short Summary)

The main point of this lab was to learn how to control things using time, especially with the `millis()` function, so the program does not stop. We built a system to control six blinking LEDs. We used a Potentiometer to choose how many of the LEDs should blink. We also used two Buttons one to arrange the speed of blinking. We made sure the button presses were clean using a debounce method. The most important part was making sure that when the speed was set to the maximum, the active LEDs stopped blinking and stayed constantly on.

2) Photos of Your Arduino Design(s)



3) Your Code (Full Sketch)

```
const int ledPins[] = {2, 3, 4, 5, 6, 7};
```

```
const int NUM_LEDS = 6;
```

```
const int POT_PIN = A1;
```

```
const int FASTEN_BUTTON = 8;
```

```
const int SLOW_BUTTON = 9;
```

```
const int NUM_SAMPLES = 8;
```

```
int lastShownValue = -1;
```

```
long blinkinginterval = 300;
```

```
long lastMillis[NUM_LEDS];
```

```
int ledstate[NUM_LEDS];
```

```
const int to_add_or_sub = 30;
```

```
const long MAX_SPEED = 500;
```

```
const long MIN_SPEED = 0;
```

```
long last_button_press_time = 0;
```

```
const long debounceDelay = 50;
```

```
void setup() {
```

```
    Serial.begin(9600);
```

```
    for (int i = 0; i < NUM_LEDS; i++) {
```

```
        pinMode(ledPins[i], OUTPUT);
```

```
        lastMillis[i] = 0;
```

```
        ledstate[i] = LOW;
```

```
    }
```

```
    pinMode(FASTEN_BUTTON, INPUT_PULLUP);
```

```
    pinMode(SLOW_BUTTON, INPUT_PULLUP);
```

```
}
```

```

void loop() {

  unsigned long currentMillis = millis();

  if (currentMillis - last_button_press_time > debounceDelay) {

    if (digitalRead(FASTEN_BUTTON) == LOW) { // if it' pressed

      if (blinkinginterval >= MIN_SPEED) {

        blinkinginterval -= to_add_or_sub; //fasten

        if (blinkinginterval <= MIN_SPEED) blinkinginterval = MIN_SPEED;

        Serial.print("fasten!");

        Serial.println(blinkinginterval);

        last_button_press_time = currentMillis;

      }

    }

    else if (digitalRead(SLOW_BUTTON) == LOW) {

      if (blinkinginterval <= MAX_SPEED) {

        blinkinginterval += to_add_or_sub;

        if (blinkinginterval >= MAX_SPEED) blinkinginterval = MAX_SPEED;

        Serial.print("slowed");

        Serial.println(blinkinginterval);

        last_button_press_time = currentMillis;

      }

    }

  }

  long rawValueSum = 0;

  for (int i = 0; i < NUM_SAMPLES; i++) {

    rawValueSum += analogRead(POT_PIN);
  }
}

```

```

}

int rawAverage = rawValueSum / NUM_SAMPLES;

int K = map(rawAverage, 0, 1023, 0, NUM_LEDS);

for (int i = 0; i < NUM_LEDS; i++) {

  if (i < K) {

    if (currentMillis - lastMillis[i] >= blinkinginterval) {

      lastMillis[i] = currentMillis;

      ledstate[i] = !ledstate[i];

      digitalWrite(ledPins[i], ledstate[i]);

    }

  } else {

    if (ledstate[i] == HIGH) {

      digitalWrite(ledPins[i], LOW);

      ledstate[i] = LOW;

    }

  }

}

if (K != lastShownValue) {

  Serial.print("Potantimeter K value: ");

  Serial.println(K);

  lastShownValue = K;

}

delay(10);

}

```

4) Your Own Scenario: New Problem (problem statement only)

- Purpose: To save energy by turning off the lights automatically when there is no one in the room.
- Inputs: PIR motion sensor (to detect if someone is in the room).
- Outputs & Expected Behavior: Light → turns ON when motion is detected and OFF after some time with no movement.

(Optional) LED or buzzer → can give a short signal before the light turns off.